

PROJECT PROPOSAL

INTEGRATED ENVIRONMENTAL MONITORING PLATFORM

Prepared By:	Mohammed Irfan KS
Roll / Reg No:	KCMIT25MCA-2011
Program:	MCA (3rd Semester)

1. Introduction and Problem Statement

Environmental conditions influence human comfort and daily activities. Indoor conditions are usually measured using local sensors, while outdoor environmental information is obtained separately from weather applications or online services. As a result, users often need to refer to different sources to understand their surrounding environmental conditions.

The Integrated Environmental Monitoring Platform is proposed to combine indoor IoT sensor readings and outdoor weather information in a single application. The platform collects data from multiple sources, processes it through a central backend, stores historical readings, and presents the information through an interactive cross-platform dashboard.

The platform provides current environmental conditions, indoor and outdoor comparison, historical graphs, and threshold-based alerts through a unified interface.

2. Aim and Objectives

Aim

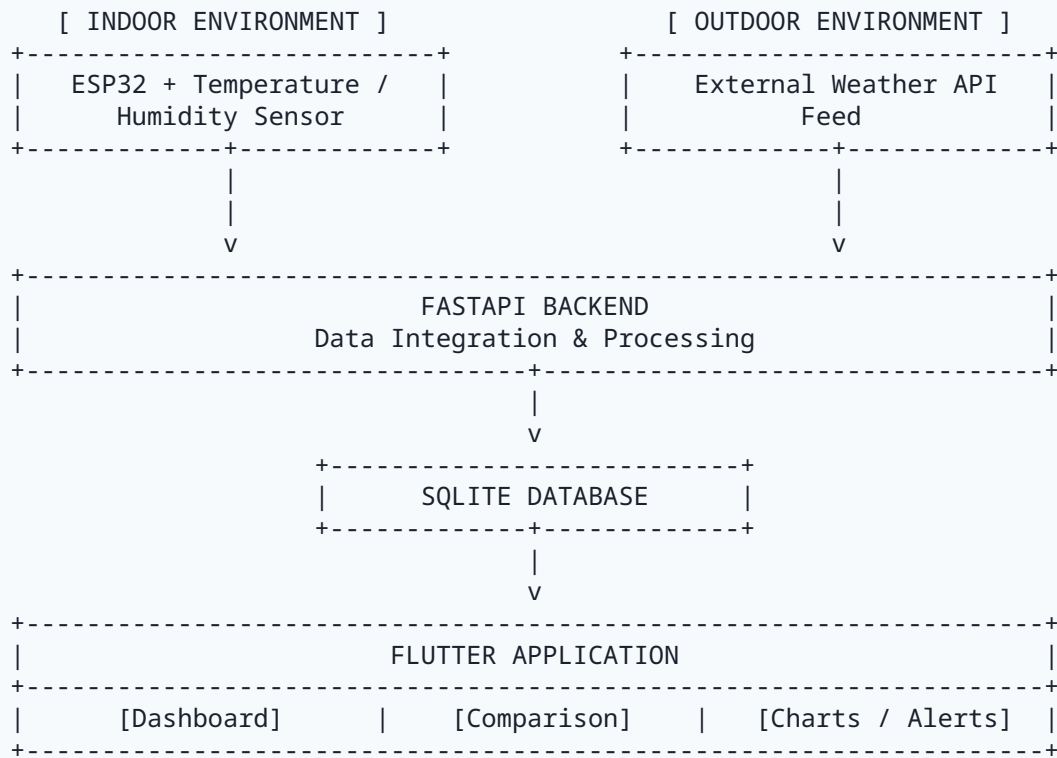
To design and develop a modular environmental monitoring platform that integrates indoor IoT sensor data and outdoor weather information and presents the data through an interactive cross-platform application.

Objectives

- Collect indoor temperature and humidity readings using an ESP32-based sensing device.
- Obtain current outdoor environmental information from an external weather API.
- Integrate data from multiple sources using a common backend application.
- Store environmental readings for historical monitoring and trend analysis.
- Develop a cross-platform Flutter dashboard for clear visualization of environmental data.
- Compare selected indoor and outdoor environmental values.
- Display alerts when environmental readings exceed predefined thresholds.
- Use a modular architecture that can support additional sensors and data sources in future versions.

3. System Architecture

The proposed platform follows a layered architecture consisting of data acquisition, backend processing, data storage, and presentation layers.



4. System Modules

4.1 Indoor Data Acquisition Module

An ESP32 development board collects temperature and humidity readings from connected sensors. The ESP32 connects to Wi-Fi and sends sensor readings to the backend using HTTP and JSON.

4.2 Outdoor Environmental Data Module

The backend obtains outdoor environmental information from an external weather API. Initial data may include outdoor temperature, humidity, wind speed, and current weather conditions.

4.3 Data Integration and Processing Module

The FastAPI backend acts as the central processing layer. It receives indoor sensor readings, obtains outdoor API data, organizes the information into a common format, and evaluates selected readings against predefined environmental thresholds.

4.4 Data Storage Module

Environmental readings are stored in an SQLite database with timestamps. Stored data is used to retrieve historical readings, generate trends, and compare indoor and outdoor conditions.

4.5 Presentation Dashboard

The frontend is developed using Flutter. It displays current condition cards, indoor and outdoor comparisons, historical charts, and environmental alert information through a responsive and interactive interface.

5. Technical Specifications and Requirements

Hardware Components

- ESP32 Wi-Fi development board
- DHT11 or DHT22 temperature and humidity sensor
- Breadboard and jumper wires
- USB power source

Software and Framework Stack

Layer	Technology
Frontend	Flutter (Dart)
Backend	FastAPI (Python)
Database	SQLite
Communication	REST API / HTTP / JSON
Outdoor Data	External Weather API
Development Tools	Android Studio / Visual Studio Code
Version Control	Git

6. Development Methodology and Sprint Milestones

The project follows an Agile and iterative development approach. Development is divided into small functional iterations so that a working version of the platform can be maintained and improved gradually.

- **Sprint 1:** Initialize the FastAPI backend and test using sample environmental data.
- **Sprint 2:** Integrate an external weather API and process outdoor environmental data.
- **Sprint 3:** Develop the basic Flutter dashboard and application navigation.
- **Sprint 4:** Design the SQLite database and implement historical data storage.
- **Sprint 5:** Add historical graphs and indoor-outdoor comparison features.
- **Sprint 6:** Integrate the ESP32 and test real indoor sensor readings.
- **Sprint 7:** Implement environmental alerts, testing, and user interface improvements.

7. System Requirements

Functional Requirements

- Receive indoor environmental readings from the IoT sensing device.
- Obtain outdoor environmental information from an external weather service.
- Store environmental readings with timestamps.
- Display current indoor and outdoor environmental conditions.
- Retrieve and visualize historical environmental data.
- Compare selected indoor and outdoor environmental values.
- Detect values that exceed predefined thresholds and display alerts.

Non-Functional Requirements

- **Usability:** The application should provide a clear and easy-to-understand interface.
- **Responsiveness:** The frontend should adapt to supported mobile, web, and desktop screen sizes.
- **Maintainability:** The platform should use a modular structure to simplify future changes.
- **Extensibility:** Additional sensors and environmental data sources should be addable with minimal structural changes.
- **Reliability:** Temporary failure of one data source should not prevent the entire platform from operating.
- **Portability:** Flutter will be used to support multiple frontend platforms from a common codebase.

8. Project Scope and Expected Outcomes

Project Scope

The initial implementation focuses on integrating indoor temperature and humidity readings with outdoor weather information. The platform includes environmental data collection, data integration, current condition monitoring, historical storage, graphical visualization, indoor-outdoor comparison, and basic environmental alerts. Advanced artificial intelligence, predictive modelling, complex cloud infrastructure, and smart device control are outside the initial project scope.

Expected Outcomes

The expected outcome is a functional full-stack environmental monitoring platform that demonstrates the integration of an IoT sensing device, an external weather API, a backend service, a database, and a cross-platform frontend application. Users should be able to view current conditions, compare indoor and outdoor values, observe historical trends, and identify environmental conditions that exceed configured thresholds.

9. Future Scope

- Indoor air quality and light intensity monitoring
- Multiple room and multiple IoT device support

- Push notifications and custom alert thresholds
- Cloud database integration and user authentication
- Advanced environmental analytics and predictive analysis
- Smart device control and automated environmental responses

10. Conclusion

The Integrated Environmental Monitoring Platform aims to provide a unified interface for monitoring environmental information from multiple sources. By combining indoor IoT sensor readings with outdoor weather data, the platform provides users with broader environmental context than a standalone sensor display or weather application.

The separation of data acquisition, backend processing, storage, and frontend presentation provides a modular structure that can be maintained and expanded in future versions. The project also provides practical experience in IoT integration, API development, database management, backend development, and cross-platform application development.